

グラフ理論 木 オイラーツアー LCA (最小共通祖先)

線形時間 LCA

1. 概要

根付き木における 2 頂点 u, v に対して, u から v へのパスが通る, 深さ最小の頂点を u, v の **LCA** といい, $\text{LCA}(u, v)$ と書きます.

LCA の計算については, オイラーツアーテクニック などでも触れました. 本講座ではそれよりも計算量オーダーの良い, Farach-Colton と Bender によるアルゴリズムについて紹介します. これは, N 頂点の木に対して, $O(N)$ 時間の事前計算のもと, LCA クエリに $O(1)$ 時間で答えるもので, したがって計算量オーダーは最適です.

2. 前提知識

スパーステーブル, オイラーツアーテクニック による LCA の計算.

3. オイラーツアーによる LCA の計算

本講座で扱うのは次の問題です.

【問題 1】

N 頂点の根付き木が与えられます. Q 個のクエリに答えてください. クエリでは 2 頂点 u, v が与えられるので, $\text{LCA}(u, v)$ を求めてください.

この問題を $O(N + Q)$ 時間で解くことが本講座の目標となります.

まずはオイラーツアーによる LCA の計算について復習します.

木のオイラーツアーをとり，訪れる頂点の深さの列を考えると，2 頂点の LCA の計算は**区間最小値クエリ**（Range Minimum Query）に帰着できるのです（以下区間最小値クエリを **RMQ** と表記します）．より詳しくは，必要となるのは最小値をとるインデックスですが，本講座の範囲ではほとんど扱いが変わらないので，これも RMQ と書くことにします．

RMQ を解く方法としては，次のようなデータ構造を用いることが代表的な手法です．

- セグメント木：事前計算 $O(N)$ 時間，クエリ $O(\log N)$ 時間．
- スパーステーブル：事前計算 $O(N \log N)$ 時間，クエリ $O(1)$ 時間．

どちらの手法もクエリまたは事前計算に $\log N$ がかってしまい，これらでは線形時間の LCA は実現できません．この計算量を改善するのが本講座のテーマです．

4. ± 1 RMQ の線形時間アルゴリズム

4.1. ± 1 RMQ

我々が解くべき RMQ は，ただの区間最小値クエリ以上の特徴を持っています．それは，隣接する 2 項の差が ± 1 のどちらかであることです．このような RMQ を **± 1 RMQ** といいます．

【問題 2】（ ± 1 RMQ）

整数列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます．ただし任意の i について

$$|A_i - A_{i+1}| = 1$$

であることを仮定します．

Q 個のクエリに答えてください．クエリでは $0 \leq L \leq R \leq N - 1$ を満たす整数 L, R が与えられるので， $A_L, A_{L+1}, \dots, A_R$ の最小値（最小値をとるインデックス）を求めてください．

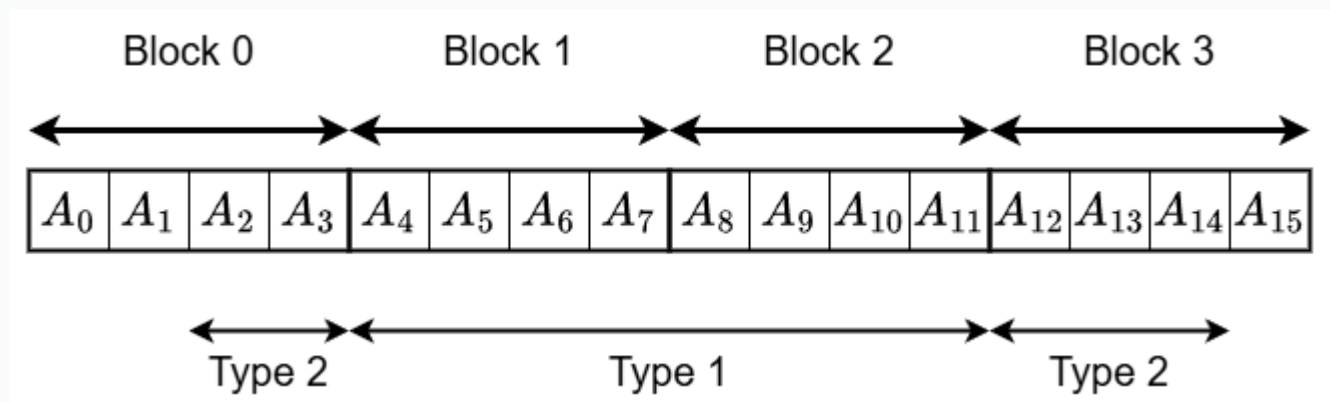
【問題 1】 を解くには，この問題を $O(N + Q)$ 時間で解ければよいということになります．

4.2. ブロック分割の利用

適当なブロックサイズ k をとり (k はあとで値を定めます), A を長さ k の連続部分列に分解します. N は k の倍数ではないかもしれませんが, 簡単のため必要であれば末尾に適当な要素を追加して, N が k の倍数であるとして説明します. $n = N/k$ とすれば, A は長さ k のブロック n 個に分割されます.

このとき, A の任意の区間は次の 2 種類の区間 3 個以下に分割できます.

- **タイプ 1**: あるブロックの先頭からあるブロックの末尾まで.
- **タイプ 2**: あるブロックに完全に含まれる区間.



よってこの 2 種類の区間に対する RMQ が解ければ, A に対する RMQ が解けることになります.

4.3. スパーステーブルの利用

j 番目のブロック内での A_i の最小値を B_j とすることで, 列 $B = (B_0, \dots, B_{n-1})$ を定めます. 列 B に対するスパーステーブルを構築しましょう. これは計算量 $O(n \log n)$ 時間で行えます.

これによってタイプ 1 の区間に対するクエリには $O(1)$ 時間で答えられるようになります.

4.4. ルックアップテーブルの利用

タイプ 2 の区間に対する RMQ について考えます.

j 番目のブロック $(A_{kj}, \dots, A_{kj+k-1})$ に対して、そのパターンを、 $A_{i+1} - A_i$ を並べてできる長さ $k - 1$ の列として定めます。

± 1 RMQ の状況において次が成り立ちます。

- パターンは高々 2^{k-1} 通りである。
- ブロック内における l 番目から r 番目の間での最小値インデックスは、ブロックパターンと l, r だけから定まる。

そこで、すべてのパターンおよび l, r に対して最小値インデックスの位置を計算し、配列に格納しておきます。この事前計算は、 $O(2^k \cdot k^2)$ 時間で行えます。

タイプ 2 の区間にクエリがきたときには、対象となるブロックのパターンから、配列の適切な値を読み取ることで $O(1)$ 時間でクエリに答えることができます。

ここで用いたような、計算結果を参照するために格納しておく配列は、しばしば**ルックアップテーブル**と呼ばれます。

4.5. 計算量解析

以上の方法で、次の計算量で ± 1 RMQ を解くことができます。

- 事前計算： $O((N/k) \log(N/k) + 2^k \cdot k^2)$ 時間。
- クエリ： $O(1)$ 時間。

$k = \frac{1}{2} \lceil \log_2 N \rceil$ などととれば、事前計算の時間が $O(N)$ 時間になり、線形時間で ± 1 RMQ を解くアルゴリズムが得られます。

4.6. 実装例

Library Checker [Lowest Common Ancestor](#) への提出です。

- <https://judge.yosupo.jp/submission/327648>

5. 関連問題

1. <https://judge.yosupo.jp/problem/lca>

6. まとめ

本講座では、Farach-Colton と Bender による LCA アルゴリズムを解説しました。

これは計算量オーダーでは最適ですが、競技プログラミングで一般的な制約下では必ずしも実行時間が最高速にならないこともあり、本講座の執筆時点では競技プログラミングで最も人気の手法というわけではありません。しかし、このような簡潔な工夫で計算量が改善できるのは非常に面白いと思います。

なお、本講座のアルゴリズムのように、「小さいブロックに分割し、ありうるパターンすべてに対するルックアップテーブルを用意する」という高速化テクニックは、**Method of Four Russians (MoFR)** と呼ばれ、他にもいろいろなアルゴリズムでの計算量改善に使用できます。AtCoder Algorithm Lectures でも何度か登場する予定です。

また、本アルゴリズムの発見者である Farach-Colton と Bender は、同様にレベル祖先問題についても簡潔な線形時間アルゴリズムを発見しています。こちらについても AtCoder Algorithm Lectures で解説する予定です。

7. 参考文献

1. Michael A. Bender, Martín Farach-Colton. The LCA Problem Revisited. Proceedings of LATIN 2000: Theoretical Informatics (LATIN 2000). pp. 88–94. 2000. DOI: 10.1007/10719839_9. https://doi.org/10.1007/10719839_9.
2. Wikipedia. Method of Four Russians. https://en.wikipedia.org/wiki/Method_of_Four_Russians. (閲覧日: 2026-03-31).
3. cp-algorithms. Lowest Common Ancestor - Farach-Colton and Bender algorithm. https://cp-algorithms.com/graph/lca_farachcoltonbender.html. (閲覧日: 2026-03-31).
4. joisino. 前処理 $O(n)$ クエリ $O(1)$ のLCAと静的RMQ. <https://joisino.hatenablog.com/entry/2017/08/13/210000>. (閲覧日: 2026-03-31).

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)

